Lab 07

HTML5 Structure and Wireframes to HTML & CSS

Overview

Lab 07 exercises:

- HTML5 Semantic Elements;
- Wireframes to HTML & CSS.

Note: Keep on top of **Weekly Journals**. A blue panel like this will tell you what lab completion evidence to provide each week.

Exercise 1: HTML5 Semantic and Structural Elements

Back in Lab 1 we first looked at how you could use HTML5 semantic tags to structure your webpage in place of the all-purpose non-semantic `<div>` tag. We have been using `<header>`, `<main>`, `<nav>`, `<section>` and `<footer>` tags throughout various lab exercises, and you probably feel more comfortable with their usage by now. Let's introduce some more semantic elements that have particular usage in the structure of different sections and grouping of content on a web page.

HTML5 Element	*Definition*
`<header>`	Used to define the header block of the HTML document, and/or header of a section.
`<nav>`	Used to define the major navigation block of a HTML document. Usually for global navigation.
`<main>` see note	Used to define the main content of the webpage. Must only be used once! *Note: This element is not supported on obsolete Internet Explorer (but only 0.6% global audience)*
`<section>`	Used to group together thematically-related content into a section in a HTML document. Each section should be identified by a heading (h1 to h6).
`<article>`	Used to define a section of self-contained independent content, like news articles, blog post or comments.
`<aside>`	Used to define content that is related to the content surrounding it, but could be considered separate.
`<footer>`	Used to define the footer block of a HTML document, and/or footer of a section.
`<address>`	Used to define contact information for an article or for the entire webpage.
`<figure>`	Represents self-contained content such as illustrations, diagrams, photos, etc., frequently with a caption - `<figcaption>`.

With all these tags, how do we choose which elements to use in various situations? Some tags seem very similar in usage. On the next page is some HTML mark-up code of most of these tags in use. After the example is small description of how they were used.

- Create new HTML document **wk7ex1.html**
- **Type** the following:

```
<!DOCTYPE HTML>
<html>
<head>
    <meta charset="UTF-8">
    <title>Your Website</title>
</head>
<body>
    <header>
        <h1>Title of Website / banner area</h1>
    </header>
    <nav>
        <ul>
            <li>menu item 1</li>
            <li>menu item 2</li>
            <li>menu item 3</li>
        </ul>
    </nav>
    <section>
        <article>
            <h2>Article title 1</h2>
            <p>Posted on June 3rd, 2015, by <a href="#">Writer</a></p>
            <p>This is the article content.</p>
        </article>
        <article>
            <h2>Article title 2</h2>
            <p>Posted on July 14th, 2017, by <a href="#">Writer</a></p>
            <figure>
                <img src="photo.jpg" width="320" height="240" />
                <figcaption>Caption related to the figure.</figcaption>
            </figure>
        </article>
    </section>
    <aside>
        <h2>Aside section</h2>
        <p>The is semi-related to the articles.</p>
    </aside>
    <footer>
        <address>Copyright. <a href="maito:name@email.com">Name</a>.
        </address>
    </footer>
</body>
</html>
```

- Take some time to look at each line of code, and how some tags are **nested** inside others.
    - Using indentations visibly shows nesting - that is, which element is the **child** element to its **parent** element. For example, in the case above `<h1>` is a child element to its parent `<header>`.

- What has each of the HTML5 tags been used for in this HTML document? Starting from the top:
    - **`<header>`** - Displays the main header / banner of the website. Although it can also be used for headers in each section as well.
    - **`<nav>`** - Displays the main navigation of the website. This could be placed anywhere (sometimes *inside* the header), as long as it identifies the global navigation.
    - **`<section>`** - Displays a section dedicated to multiple article postings. You can have many sections with different themes.
    - **`<article>`** - As seen it is used twice in the example, and displays independent posts by an author called “writer”. Independent means that each article can be read separate from another article, like two separate blog posts.
    - **`<figure>`** - Displays an image, with a caption through the use of **`<figcaption>`**. The figcaption tag needs to be inside the figure to display the caption directly beneath the figure (which does not necessarily need to be an image).
    - **`<aside>`** - Displays another section that is related to the content surrounding it, but still considered separate. Quite often this is placed to the left or right of the related content.
    - **`<footer>`** - Displays the main footer of the website. Although it can also be used for footers in each section as well.
    - **`<address>`** - Identifies the contact email on the website. Note that its default font-style is in italics.
    - ***Note** about **`<section>`** and **`<article>`**. The interesting thing about these tags is that a section could have many articles, and an article may have many sections. Both are acceptable; it depends on the content the web developer/author wishes to portray.*
    - **`<main>`** - We did not use main in this example, but it could easily have been used in place of section or to wrap the entire webpage.

- **Open** the file in **Google Chrome**.
- Without any styles (or the actual image), the webpage is pretty plain, but the tags can easily be styled with CSS selectors to set them up with their intended usage.
- Test this HTML page also in **Internet Explorer 9.0+** and **Mozilla Firefox**.
- Are they any different? No? Good, this set of HTML5 tags have great compatibility with the global market share of browsers that are used.
- With this knowledge of HTML5 page structure through semantic tags, let’s move on to the second exercise.

Exercise 2: Creating an HTML/CSS Webpage from a Wireframe

In the last lab you created your digital wireframes in specific software (Pencil) that could output an image file. This exercise will show you how to use a wireframe image as a reference for creating a webpage with HTML and CSS.

The wireframes we created displayed most of the information you are going to need to build the associated webpage, such as fonts, sizes, colours, content positions and dimensions. We will start with the example wireframe, and then you can apply what you have learnt to your own wireframes.

- Let’s setup an ordered folder structure:
    - **Create a folder** called **Lab 7**.
    - Inside **Lab 7** folder, **create** three more folders called **css**, **images** and **fonts**.

a. Optimising and Preparing the Images

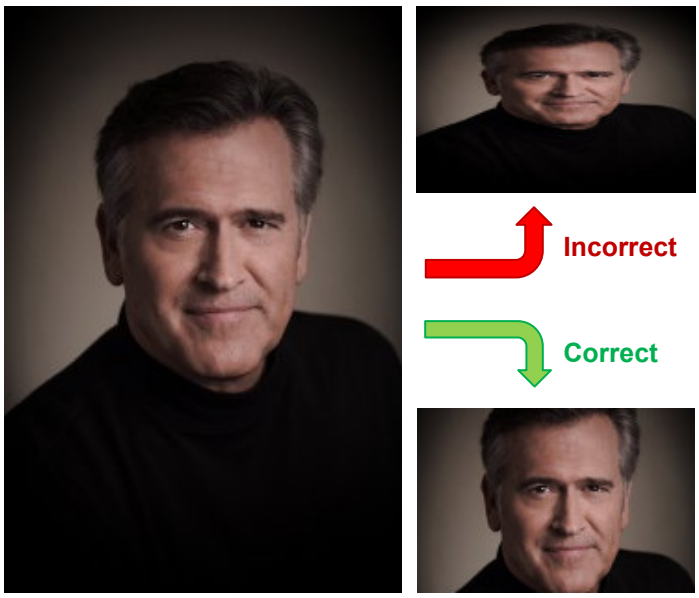
Before using images on a website, you should **always** optimise them.

If you did this in the last lab for the example wireframe, you will have already created and exported optimised images and can skip to step b. If you did not, you should continue below to optimise the images.

- **Question:** Why optimise images for a website?
    - **Quick Answer:** It will load faster.
    - **Long Answer:** Imagine this scenario – photos taken on an 8 Megapixel smartphone (*quite low by today's standards*) as an example. The image resolution of photos from these smartphones is 3264 x 2448 pixels. If you were to use these images on your website without any resizing or cropping, the images would be considerably larger than you need. An image of this resolution would take anywhere between 1mb to 3mb, a whole lot of unnecessary data for the user to download (especially when you only need a much smaller image).
- You will need the **Lab 07 Files** zip file from Moodle. **Extract** the zip so you can edit the images.
- Open **GIMP** (the following instructions are for GIMP, but you may adapt to another Image Editor).
- Drag the 4 images (**imageleft.jpg**, **imagemiddle.jpg**, **imageright.jpg**, and **logo.jpg**) into the GIMP window. Or open them in GIMP one at a time. If asked, **convert** the colour profiles.
- Looking at the example wireframe, the dimensions of:
    - **logo.jpg** should be 150 x 100.
    - **imageleft.jpg** should be 340 x 200.
    - **imagemiddle.jpg** should be 340 x 406.
    - **imageright.jpg** should be 340 x 200.
- Let's start with **logo.jpg**.
- You could use the **Image > Scale Image** command and set the dimensions to the correct width and height, but what would happen? The images would skew (stretch and/or squash) to make the images look terrible. See **"incorrect"** image to the right →
- Use **Image > Scale Image** command. Make sure the **chain is linked** (this will lock aspect ratio and prevent skewing)
- **Set** the **width only** to the correct width of **150** pixels. The height will still be too large, that is fine we can fixed it, but if it is too small, we cannot scale it larger without loss of quality.
- Click Scale.
- The locked aspect ratio automatically sets height to 225 pixels. So far so good, as the image is not skewed. But now we need to crop the image down to 100 pixels height.
- The easiest way is to use **Image > Canvas Size**.
- Do this now, and **set** the **canvas width** to **150**, and **height** to **100** pixels (the chain must be unlinked), and hit **enter**. **Don't click "Resize" yet.**

Optimising Images

- **Dimensions:** the width and height should match its space on the webpage.
- **File Size:** export to PNG if your image requires transparency. JPG if your image does not.
- **Aspect Ratio:** You **do not** want skewed, stretched or squashed images. They look unprofessional.



- Now **move** the face position to set the box as the area of the face you want to keep for the logo →
- **Click Resize**. It should now look like the **"correct"** image on the previous page.
- File > Export As…
- **Set** it to **JPG** (if it isn't already), keep it **named logo.jpg**, and choose to save it in a safe location.
- When is asks, **set the quality to 90**. This ensures pretty clear images but with some additional compression.
    - **The file is now only 13.2kb, down from 398kb.** This is a massive difference, and also remember a photo straight out of a camera or smart phone will take **mega**bytes, not **kilo**bytes of data.
- **Optimise** the remaining three images we need for this webpage mock-up. Make sure they end up the correct size listed above, and that their aspect ratio remains the same and is not skewed.

b. Preparing the Layout

- **Open** Notepad++ and **save** an HTML file as **wk7ex2.html** inside the "**Lab 7**" folder.
- Also **save a new** document as CSS called **wk7ex2.css** inside the "**Lab 7\css**" folder.
- Place the **3 font files** (from Lab 07 files) inside the "**Lab 7\fonts**" folder
- Having placed the files (and the images previously) in their appropriate folders, you will be able to use ***relative addressing*** when linking to any of these files.
    - *If you are unsure what relative addressing means, do some research on the Internet to get up to speed. Keep in mind that without relative addressing your website will not work on any other computer than the one you created the website on!*
- Start with your HTML file with a basic structure as below:

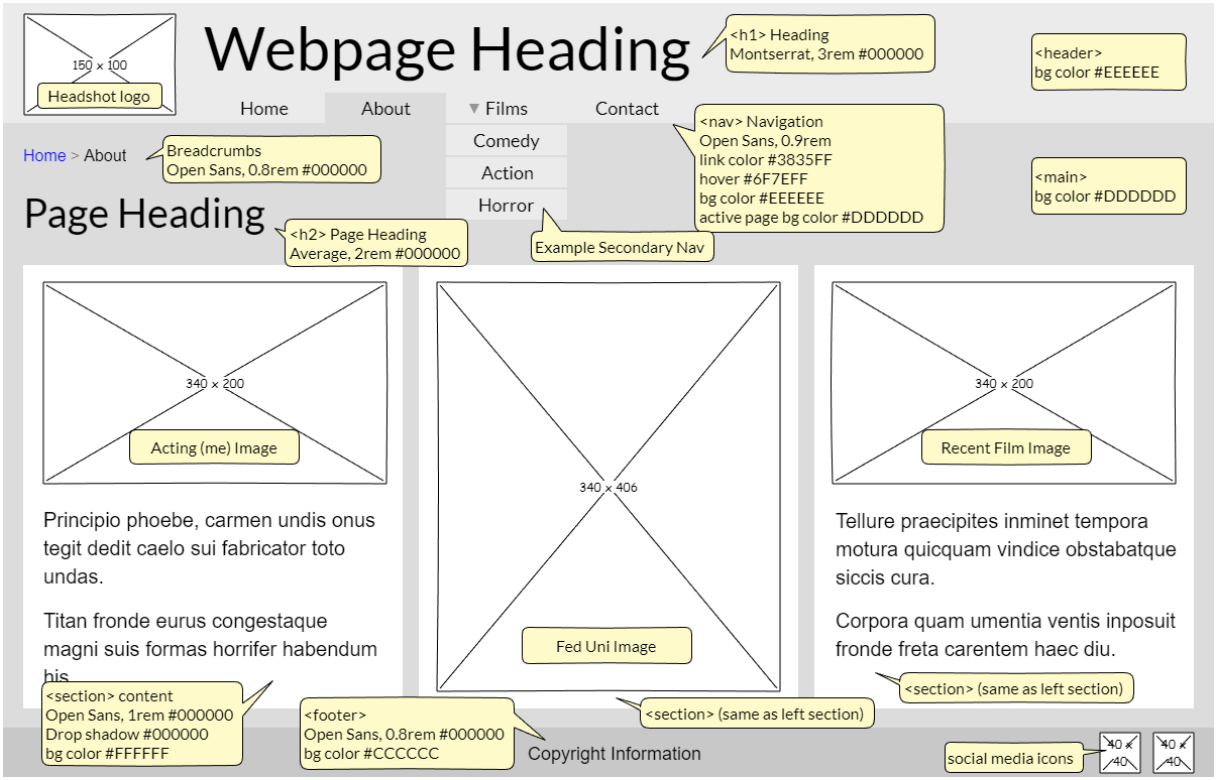
```
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Week 7 Exercise 2</title>
</head>
<body>

</body>
</html>
```

- *The !DOCTYPE declaration is to declare that HTML5 is to follow. It should always be the first line of code.*
- *The meta tag with charset attribute set to UTF-8 allows your webpage to encode any characters in the Unicode standard and is backwards compatible with ASCII. It should be the first tag placed inside the head.*
- *Now that you know this, you should always place these two within your HTML document.*
- Moving on, we can already begin to enter some code into the HTML file. The **CSS file** needs to be **linked** and the easiest way to do this is via entering the following code into the **head** below the title:

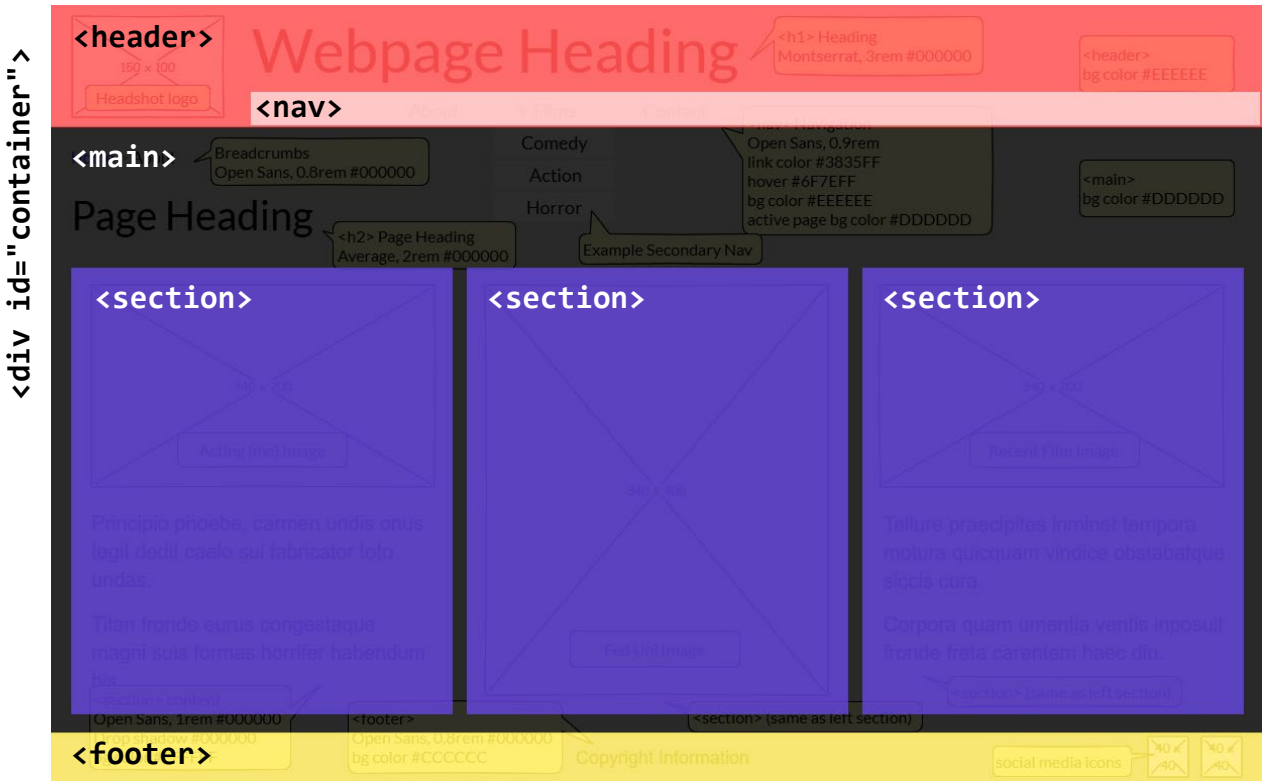
```
<link href="css\wk7ex2.css" rel="stylesheet">
```

- Note the **relative addressing**. We are telling the browser to look inside the **css** folder for the **wk7ex2.css** file.
- On the next page, we plan how to structure the webpage.

- Based on the example wireframe:



- We can decide how to divide the content into separate blocks. Each block is identified via the HTML5 tag or a div if no HTML5 tag is appropriate. The div tags show the ID we will assign to it, as shown below. If the sections were going to be styled differently, they could each have their own ID as well, but in this case they are all similar.



- With the plan above of the structure, **enter** the HTML code to match this structure:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Week 7 Exercise 2</title>
    <link href="css\wk7ex2.css" rel="stylesheet" />
</head>
<body>
    <div id="container">
        <header>
            Red Box - Logo and Heading
            <nav>
                White in Red Box - Navigation
            </nav>
        </header>
        <main>
            Black Box - Breadcrumbs and Sub-Heading
            <section>
                Blue Box 1 - Image and Text
            </section>
            <section>
                Blue Box 2 - Large Image
            </section>
            <section>
                Blue Box 3 - Image and Text
            </section>
        </main>
        <footer>
            Yellow Box - Footer
        </footer>
    </div>
</body>
</html>
```

A few notes:

- We placed everything inside the body in a `<div id="container">` in order to control the position of the entire page with the container CSS selector.
- The `<nav>` was placed inside the header as this is how the wireframe portrays it.
- The three `<section>`s are placed inside a `<main>` to indicate the main content area of the page and to provide a nice division from header and footer.

c. CSS Reset and Initial Setup

- Before we begin to code our CSS it is necessary to make sure that all the default margins of every selector we will be using is set to zero. Failing to do this will ultimately give us headaches once we start to create many more pages.
- The code that will handle this is as follows:

```
/* CSS Reset */
* {
    margin: 0;
    padding: 0;
}
```

- ***A word of caution:*** *we are using the wildcard character * here, which is expedient but not always the best way. If you decide to continue with web design, you are advised to create a Reset CSS file that contains your most frequently used selectors that need resetting to avoid situations where you find yourself constantly changing these resets. Web designers, like programmers, eventually arrive at a style entirely their own, using settings that seldom change and so their CSS Reset may as well contain THEIR defaults.*

- Update the CSS file with the following code:

```
html {
    font-size: 1.25em;
}

body {
    background-color: #ffffff;
    font-family: Open Sans, sans-serif;
    color: #000000;
}
```

- **Test** the code in your browser to make sure that everything works at this point. If you don't see a page where each HTML element is lined up after another using a sans-serif font-face, then something has gone wrong, and you need to check your code.
    - **NOTE:** *The html selector was originally set to 1.25em. What this means is the root element (html) is 1.25 times larger than the default font-size (default font-size is always 16px), which equals 20px.*
- Now you are ready to do some serious coding.
- Since the wireframe was set to be 1200 pixels wide, we will set up the CSS for the **#container** id selector to reflect this. Centering of the div will be done via the margin attribute. **This translates into the following code**:

```
#container {
    width: 1200px;
    margin: auto;
    background-color: #dddddd;
}
```

- **Save** and **test**. You should now see the grey 1200 pixel width "container" centered in the screen with white space on either side.

d. Creating the <header>

- For the **<header>** section, we will place the logo image inside a div with an **id** named **"logo"**. This will allow us to float this entire div to the left allowing other elements (such as the h1 heading and the navigation) to be placed to the right of it.
- The **HTML for the header** is as follows:

```
<header>
	<div id="logo">
		<img src="images/logo.jpg" alt="Bruce Campbell Headshot">
	</div>
	<h1>Bruce Campbell</h1>
	<nav>
		White in Red Box - Navigation
	</nav>
</header>
```

- And the CSS for the header is:

```
header {
    width: 1190px;
    height: 100px;
    padding: 5px;
    background-color: #eeeeee;
}

header #logo {
    width: 150px;
    height: 100px;
    float: left;
}

header h1 {
    font-family: Montserrat, sans-serif;
    font-size: 3rem;
    margin-left: 170px;
}
```

- Notice in the `header` selector we set the width to 1190px and not 1200px? That is because the padding takes up 5 pixels all around the whole header.
- The `h1` heading inside the header was set to 3rem. This means it will be 3 times larger than the root element (the html selector). 3 times 20px = 60px. However, we are using em and rem for easier scalability. These values are always relative to another value, whereas directly using pixels is a fixed value!
- Also, within the h1 selector you can see we set the `margin-left` to 170 pixels, otherwise it would have sat right against the logo image.
- **Save** and **test**.

e. Embedding Fonts

Not all users that visit your website will have some fonts such as Montserrat, Open Sans, and Average installed on their operating system, so the browser will replace them with either the default sans-serif or serif font for that operating system. This is a safe fallback option, but not ideal, it will not appear as intended and possibly mess with text fitting the desired content blocks.

There are two preferred methods for embedding fonts:

1. A copy of the font stored with the website, referenced via **CSS3 @font-face**
2. An API like **Google Fonts** or **Adobe Fonts**

We will use **@font-face** method today, as Google is blocked in some countries. If you are building a website and embedding fonts, be sure you know your target audience geographic location, and you use a suitable font embedding method.

- You should have already stored the fonts provided in Lab 07 files in the Lab 7/fonts folder. We will access them next.
- At the **very TOP** of the **CSS** file (above the CSS reset), **add the following code**:

```
@font-face {
  font-family: "Montserrat";
  src: url("../fonts/Montserrat-Medium.ttf") format("truetype");
  font-style: normal;
  font-weight: 500;
}

@font-face {
  font-family: "Average";
  src: url("../fonts/Average-Regular.ttf") format("truetype");
  font-style: normal;
  font-weight: 400;
}

@font-face {
  font-family: "Open Sans";
  src: url("../fonts/OpenSans-Regular.ttf") format("truetype");
  font-style: normal;
  font-weight: 400;
}
```

- Notice the weightings? The Montserrat medium font provided was of weight 500, the other two fonts were regular fonts of weight 400. Giving the weight is not necessary, but the browser may apply the wrong weighting automatically.
- Now to use these fonts we can just specify the font-family property.
- If you **Save and Refresh**, you should notice at least the body content changes font from before, as it is rare that people have Open Sans installed unless you have used it for other projects.
    - But now without it installed, the browser can access the font file specified in the CSS.

f. Creating the <nav> inside the <header>

- For the **main global navigation**, we will try to create the tabbed appearance of the wireframe with a drop-down menu on the Films category. So, the menu will be created in blocks with hover over effects. It will start as an unordered list, but with the list-style set to none.
- **Type** the following code in your HTML document replacing the current **nav** content:

```
<nav>
    <ul>
        <li><a href="index.html">Home</a></li>
        <li id="selected"><a href="about.html">About</a></li>
        <li><a href="films.html">Films</a>
        <ul>
            <li><a href="comedy.html">Comedy</a></li>
            <li><a href="action.html">Action</a></li>
            <li><a href="horror.html">Horror</a></li>
        </ul>
        </li>
        <li><a href="contact.html">Contact</a></li>
    </ul>
</nav>
```

- **Save** and **test**.
    - With no CSS applied to the nav selector, it will just create a list overlapping the main block.
- In the **CSS** for the **nav selector** first we set the margin properties:
    - Similar to the heading, the `margin-left` is set so the navigation sits a little further out from the image.

```
nav {
    margin-left: 170px;
}
```

- Next we **set** the styles for the unordered lists inside the navigation (we use the nav selector followed by its child element - the ul, so if you were to create any additional unordered lists, the styles with the nav selector won't apply to them):

```
nav ul {
    list-style: none;
    position: relative;
    float: left;
}

nav ul a {
    display: block;
    color: #3835ff;
    text-decoration: none;
    line-height: 1.6rem;
    padding: 0 30px;
    font-size: 0.9rem;
    text-align: center;
}
```

```
nav ul li {
    position: relative;
    width: 119px;
    float: left;
}

nav ul ul {
    display: none;
    position: absolute;
    top: 100%;
    left: 0;
    background: #eeeeee;
}

nav ul ul li {
    float: none;
    width: 119px;
    z-index: 100;
}

nav ul ul a {
    line-height: 0.9rem;
    padding: 10px 15px;
}
```

- **Save** and **test**.
    - With the unordered list and list item styles defined, a nice touch is to define the currently active webpage's menu item link with a unique appearance. We are making an About page, so we want to highlight that this page is currently active – this way the user can easily see which page they are on.
- **Add the code** below to the bottom of your **CSS**:

```
nav ul li#selected {
    background: #dddddd;
}
```

- Next, we **need to define what happens when a user hovers** over each navigation link with their mouse:

```
nav ul li:hover {
    background: #ffffff;
}

nav ul li a:hover {
    color: #6f7eff;
}

nav ul li:hover > ul {
    display: block;
}
```

- **Save** your **HTML** and **CSS** files and **test**.

- The header should look complete, and the navigation should work have hover effects, including a drop-down menu on the “Gallery” link (but clicking the links takes you nowhere as no other pages are made).

g. Creating the Main Content Area

- The **main content area** is surrounded by a **<main>** according to the original plan. The wireframe for this shows some padding, between the edge and the actual content. Let’s first style this id selector in the CSS:

```
main {
    padding: 15px;
    height: 570px;
}
```

- The height was set so that later on when you get to the footer, it will follow on exactly after the bottom of the main.
- Now in the HTML, replace the first piece of placeholder text inside the **<main>** element with:

```
<p id="breadcrumbs"><a href="index.html">Home</a> > About</p>
```

- **Save** and **test**.
- Back to the CSS, we **need** to setup styles for the paragraph with the newly created **"breadcrumbs"** id. Why did we choose an id here, and not a class? Because this style will not be used on any other elements.
- Now to the **#breadcrumbs** id selector in the CSS:

```
#breadcrumbs {
    font-size: 0.8rem;
}

#breadcrumbs a:link{
    text-decoration: none;
}

#breadcrumbs a:hover {
    text-decoration: underline;
}
```

- Next up is the sub-heading for the page title. Easy, **use a <h2> inside the HTML** file after the breadcrumb paragraph.

```
<h2>About Bruce</h2>
```

- **Do this**, and then **save** and **test**.
    - The heading is close to the bottom edge of our breadcrumbs. This is because we reset all margins and padding at the start of the CSS file, removing all defaults.

- **You can fix the <h2>**, by applying styles to an h2 selector in the CSS. While you are at it, **add** the font-family and font-size:

```
h2 {
    margin-top: 20px;
    margin-bottom: 20px;
    font-family: Average;
    font-size: 2rem;
}
```

- Next, we need to **setup the three** section **selectors**. **Return to the bottom of your CSS file** and **enter** the following:

```
section {
    margin: 10px;
    padding: 15px;
    width: 340px;
    height: 600px;
    float: left;
    background-color: white;
}
```

- How did we come to these values?
    - Look at the **<main>** that all of the **<section>**s are placed in. It has a padding of 15 pixels, which applies to each edge of this block: top, bottom, left and right. So, the initial page width of 1200 pixels is reduced by 30 pixels.
    - That leaves us 1170 pixels for three equally wide sections. 1170 divided by 3 equals 390 pixels.
    - Setting the width of each section to 340 pixels for the image leaves 50 remaining.
    - We can use these 50 pixels to provide an all-around margin of 10 pixels and padding of 15 pixels.
- Moving on, fill the **<section>**s with their content! **Edit** the **HTML page** and the **<section>** tags to:

```
<section>
    <img src="images/imageleft.jpg" alt="Bruce Profile Image">
    <p>Lorem ipsum dolor sit amet</p><br>
    <p>Consectetur adipiscing elit, sed do eiusmod tempor incididunt ut
    labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud
    exercitation ullamco.</p>
</section>
<section>
    <img src="images/imagemiddle.jpg" alt="Fed Uni Mt Helen Campus">
</section>
<section>
    <img src="images/imageright.jpg" alt="Recent Film Image">
    <p>Lorem ipsum dolor sit amet consectetur adipiscing elit, sed do
    eiusmod.</p><br>
    <p>Tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim
    veniam, quis nostrud.</p>
</section>
```

- **Save** and **test**.

- Perhaps the text is sitting too close to the images. **Add** a `margin-top` in the CSS to the paragraphs in the sections:

```
section p {
    margin-top: 5px;
}
```

- OK, the content is completed. The only thing remaining is the Footer.

h. Creating the <footer>

- First you should replace the placeholder content with the following:

```
<p>Copyright. All Rights Reserved. Bruce Campbell.</p>
```

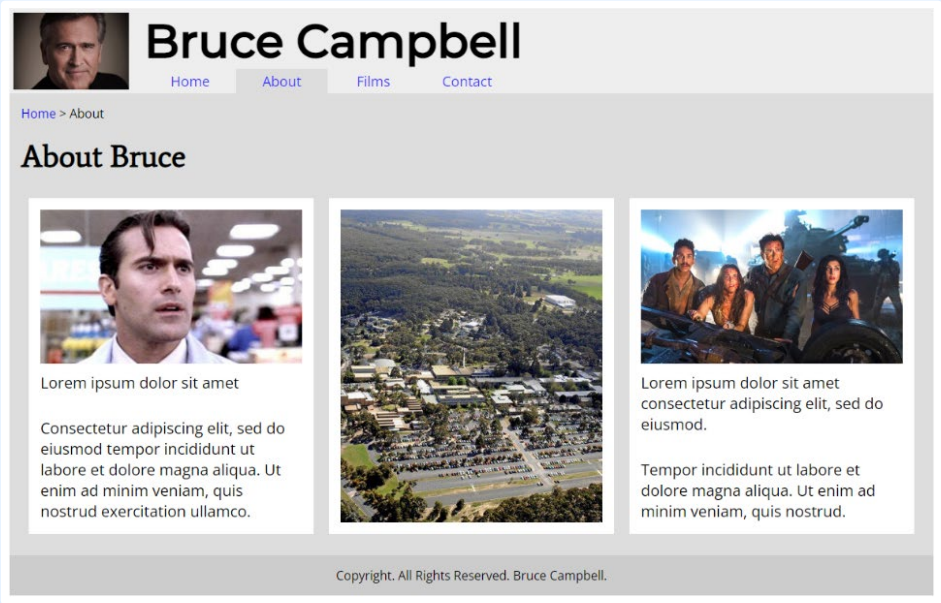
- There is not much to the footer styles. **Apply a couple of styles** to the `footer` selector in the CSS:

```
footer {
    padding: 15px;
    background-color: #cccccc;
    font-size: 0.8rem;
    text-align: center;
}
```

- **Save** and **test**.

Weekly Journal: Lab Completion Evidence

- As usual, the example → is a little bland, due to minor greyscales.
- **Modify the CSS** with your own **colour design**, place your **own name** at the top, and change the **images** to your own choice.
- **Save your file again**.
- **Take a screenshot** of your finished webpage!
- **Add this screenshot to your Weekly Journal 5 to 10 (word) document for Lab 7.**



i. (optional) Extra Tasks

- From this point see if you can figure out these challenges:
    - **Edit the HTML and CSS to add** the **drop-down arrow next to Films category,** and the **two social media icons** of your choice to the **footer,** as the original wireframe indicates.
    - What happens if you add another `<section>` after the current ones? **Try it!**
        - **How would you fix this** if you wanted to expand to have six `<section>`s?
- After you finish, **save** and **test** the webpage in the browser.